Name : P. Samanvith
Batch No : 40
HT No : 2303A52090

Task 1 – Python to Java Conversion

Task Description: Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance. Ensure proper handling of file streams, exceptions, and input/output formatting.

Prompt Used for AI Assistance: "Convert the following Python script that writes the text 'Hello, AI Assisted Coding!' to a file named 'sample.txt' and then reads it back into the console into equivalent Java code. Ensure proper handling of file streams, exceptions (try-catch blocks), and console output formatting."

Original Python Code:

```
# Open file in write mode and write text
with open('sample.txt', 'w') as file:
    file.write("Hello, AI Assisted Coding!")
    print("Successfully wrote to the file.")

# Open file in read mode and print content
with open('sample.txt', 'r') as file:
    content = file.read()
    print("File content:", content)
```

Converted Java Code & Explanation:

```
import java.io.File;
import java.io.FileWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class FileHandling {
    public static void main(String[] args) {
        String fileName = "sample.txt";
        String textToWrite = "Hello, AI Assisted Coding!";

        // Write to file using FileWriter inside a try-with-resources block
```

```
        try (FileWriter writer = new FileWriter(fileName)) {
            writer.write(textToWrite);
            System.out.println("Successfully wrote to the file.");
        } catch (IOException e) {
            System.out.println("An error occurred while writing.");
            e.printStackTrace();
        }

        // Read from file using FileReader and BufferedReader
        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
            String content = reader.readLine();
            System.out.println("File content: " + content);
        } catch (IOException e) {
            System.out.println("An error occurred while reading.");
            e.printStackTrace();
        }
    }
}
```

Explanation: * **Syntax & Libraries:** Python uses a simple built-in open() function with the with statement for automatic resource management. Java requires importing java.io.* classes and using explicit try-catch blocks (or try-with-resources) to handle IOException gracefully.

- **Execution:** Java uses FileWriter to create/write to the file and BufferedReader wrapping a FileReader to read text efficiently.

Expected Output:

Successfully wrote to the file.
File content: Hello, AI Assisted Coding!

Task 2 – C++ to Python Function Conversion

Task Description: Translate a C++ program that implements a sorting algorithm into Python using AI assistance. Ensure logical correctness and validate results using multiple test cases.

Prompt Used for AI Assistance: "Translate this C++ implementation of the Bubble Sort algorithm into an equivalent Python script. Ensure logical correctness, validate results using an unsorted array of integers, and maintain the exact sorting behavior."

Original C++ Code:

```cpp
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Converted Python Code & Explanation:

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n - 1):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Pythonic swap

# Test Cases
test_case_1 = [64, 34, 25, 12, 22, 11, 90]
print("Original Array:", test_case_1)
bubble_sort(test_case_1)
print("Sorted Array:", test_case_1)
```

Explanation: * **Syntax & Libraries:** C++ uses explicit type declarations (int arr[], int n) and a temporary variable to perform swaps. Python infers types and does not require explicit length passing since len(arr) is a built-in function. Python also supports direct tuple unpacking for swapping variables (a, b = b, a).

- **Execution:** Both maintain an $O(n^2)$ time complexity. Python utilizes the built-in range() function for the inner and outer loops.

Expected Output:

Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]

Task 3 – SQL to Pandas Query

Task Description: Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Prompt Used for AI Assistance: "Convert the following SQL query that calculates total sales per department into equivalent Pandas DataFrame code. Provide a sample dataset using a dictionary to test the code and print the resulting grouped DataFrame. SQL: SELECT department, SUM(sales) as total_sales FROM sales_data GROUP BY department;"

Original SQL Query:

```
SELECT department, SUM(sales) as total_sales
FROM sales_data
GROUP BY department;
```

Converted Python (Pandas) Code & Explanation:

```
import pandas as pd

# Sample dataset
data = {
   'department': ['IT', 'HR', 'IT', 'Sales', 'HR', 'Sales', 'IT'],
   'employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace'],
   'sales': [1200, 800, 1500, 2000, 500, 2500, 1000]
}

# Load data into a Pandas DataFrame
df = pd.DataFrame(data)

# Equivalent Pandas Operation
# Group by 'department', select 'sales' column, sum it, and reset index to match SQL output
```

```
result_df = df.groupby('department')['sales'].sum().reset_index()

# Rename the column to match the SQL alias "total_sales"
result_df = result_df.rename(columns={'sales': 'total_sales'})

print("Original DataFrame:\n", df)
print("\nPandas Aggregation Output:\n", result_df)
```

Explanation: * **SQL:** Uses declarative syntax to group and sum.

- **Pandas:** Uses procedural method chaining. The groupby('department') method matches SQL's GROUP BY. ['sales'].sum() corresponds to SUM(sales). Finally, .reset_index() is used to convert the resulting Series back into a flat DataFrame format, similar to how standard SQL tables are displayed.

Expected Output:

```
Original DataFrame:
  department employee  sales
0         IT    Alice   1200
1         HR      Bob    800
2         IT  Charlie   1500
3      Sales    David   2000
4         HR      Eve    500
5      Sales    Frank   2500
6         IT    Grace   1000

Pandas Aggregation Output:
  department  total_sales
0         HR         1300
1         IT         3700
2      Sales         4500
```

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Task Description: Select a practical real-world application requiring the same logic in two different programming languages. Execute and test both versions to ensure identical functionality. Document differences observed.

Scenario Selected: Converting a simple REST API endpoint that returns a JSON message from Python (Flask) to Java (Spring Boot).

Prompt Used for AI Assistance: "Convert this lightweight Python Flask REST API that has a single GET endpoint returning JSON {"message": "Hello from API!"} into an equivalent Java Spring Boot REST Controller. Also, provide documented observations on the differences in syntax, frameworks, and execution behavior."

Original Python (Flask) Code:

```
from flask import Flask, jsonify

app = Flask(__name__)

# Define a simple GET endpoint
@app.route('/api/greeting', methods=['GET'])
def greeting():
    return jsonify({"message": "Hello from API!"})

if __name__ == '__main__':
    app.run(port=5000)
```

Converted Java (Spring Boot) Code:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.HashMap;
import java.util.Map;

@SpringBootApplication
@RestController
public class ApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(ApiApplication.class, args);
    }

    // Define a simple GET endpoint
    @GetMapping("/api/greeting")
```

```
    public Map<String, String> greeting() {
        Map<String, String> response = new HashMap<>();
        response.put("message", "Hello from API!");
        return response; // Spring Boot automatically converts Maps/Objects to JSON
    }
}
```

Documented Observations:

1. **Syntax & Boilerplate:** Python's Flask framework is incredibly lightweight and requires minimal boilerplate code; an API can be spun up in 5 lines. Java's Spring Boot requires standard class definitions, a main application class, and specific imports.
2. **Libraries & Annotations:** Flask uses @app.route decorators. Spring Boot relies heavily on dependency injection and annotations (@SpringBootApplication, @RestController, @GetMapping) to route HTTP requests automatically.
3. **Data Handling:** In Flask, Python dictionaries are explicitly converted to JSON strings using jsonify. In Spring Boot, the Jackson library running behind the scenes automatically serializes Java objects (like Map or custom classes) into JSON responses.
4. **Execution Behavior:** Flask runs via a lightweight built-in WSGI server for development, compiling on the fly. Spring Boot runs inside an embedded Tomcat server and requires ahead-of-time compilation of the Java application before execution.

Expected Output (for both, when navigating to http://localhost:5000/api/greeting or http://localhost:8080/api/greeting respectively):

```
{
  "message": "Hello from API!"
}
```

Task 5 – Basic Function Translation (Python to C++)

Task Description: Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance. Test with at least three different input values.

Prompt Used for AI Assistance: "Translate this simple Python function def check_even_odd(num): return 'Even' if num % 2 == 0 else 'Odd' into a complete, executable C++ program. Include a main function that tests the logic with at least three different input values (e.g., 4, 7, and 0), and outputs the results to the console."

Original Python Code:

```
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

# Testing the function
for val in [4, 7, 0]:
    print(f"{val} is {check_even_odd(val)}")
```

Converted C++ Code & Explanation:

```
#include <iostream>
#include <string>

using namespace std;

// C++ equivalent function
string checkEvenOdd(int num) {
    if (num % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

int main() {
    // Array containing at least three test inputs
    int testValues[] = {4, 7, 0};

    // Loop through test values and print output
    for (int i = 0; i < 3; i++) {
        cout << testValues[i] << " is " << checkEvenOdd(testValues[i]) << endl;
    }

    return 0;
}
```

Explanation: * **Syntax:** The C++ function requires an explicit return type (string) and parameter

type (int). The modulo operator (%) logic remains identical across both languages.

- **Input Handling & Output:** Python uses f-strings for formatted output (print(f"...")), whereas C++ utilizes the cout stream from the <iostream> library. We used an array to store the test cases and a standard for loop to iterate over them and execute the translated function.

Expected Output:

4 is Even
7 is Odd
0 is Even